

# DOSSIER D'ARCHITECTURE LOGICIELLE

Cartographie des Données (PostgreSQL) - Architecture V2

## 1. Objectif et Topologie de la Base de Données

Lors de la transition vers l'architecture V2, le modèle de données (MCD) a subi une mutation profonde. Pour répondre aux exigences d'un SI (Système d'Information) d'entreprise, la base de données monolithique a été transformée en une base de données fédérée, hébergeant simultanément le modèle métier et les schémas certifiés des moteurs d'intégration (Keycloak et Flowable).

Le cluster PostgreSQL 16 héberge aujourd'hui très exactement **190 tables** interconnectées, assurant une séparation stricte des préoccupations (Separation of Concerns).

## 2. Répartition Exacte du Schéma Relationnel



### 3. Analyse Détaillée des Familles de Données

|                                             |                                                                                                                                                                                                                                                                                                                                                                                                            |
|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Domaine Sécurité<br/>(Keycloak)</b>      | <b>Exemples :</b> <code>user_entity</code> , <code>realm</code> , <code>client</code> , <code>role_entity</code><br>Gère la cryptographie, le stockage des utilisateurs, les sessions JWT, les politiques de mot de passe et l'arborescence des rôles composites (RBAC). Cette externalisation garantit la conformité aux normes de sécurité (ex: non-stockage des mots de passe dans l'espace métier).    |
| <b>Domaine Orchestration<br/>(Flowable)</b> | <b>Exemples :</b> <code>act_ru_execution</code> , <code>act_re_deployment</code> , <code>act_hi_taskinst</code><br>Le moteur génère ses propres tables de <i>Runtime</i> (RU), de <i>Repository</i> (RE) et d' <i>History</i> (HI). L'historisation native (Audit Log) est entièrement couverte par les 25+ tables de la série <code>act_hi_*</code> , rendant tout développement d'audit manuel obsolète. |
| <b>Domaine Métier<br/>(IncidentFlow)</b>    | <b>Exemples :</b> <code>incidents</code> , <code>comments</code> , <code>attachments</code><br>Le périmètre de développement se restreint exclusivement à la valeur ajoutée métier. Les tables locales de gestion d'utilisateurs et de permissions de la V1 ont été supprimées du code source.                                                                                                             |

### 4. Intégrité et Clé Étrangère Conceptuelle

Afin de garantir l'intégrité entre ces trois familles isolées, l'architecture repose sur des **clés étrangères conceptuelles** (Soft Foreign Keys). L'entité métier `incidents` référence directement le cycle de vie BPMN via l'attribut `process_instance_id`, et référence l'auteur via un identifiant unique (UUID ou Email Keycloak).

### 5. Argumentaire Architectural

*"Dans une application monolithique standard, la complexité de gestion incombe au code source métier. Dans notre architecture V2, la base de données contient 190 tables, mais la force du modèle est que la responsabilité de l'équipe de développement se limite à la maintenance des 5 tables métier. Les 185 autres tables sont maintenues, optimisées et sécurisées de manière totalement transparente par les moteurs standards de l'industrie."*